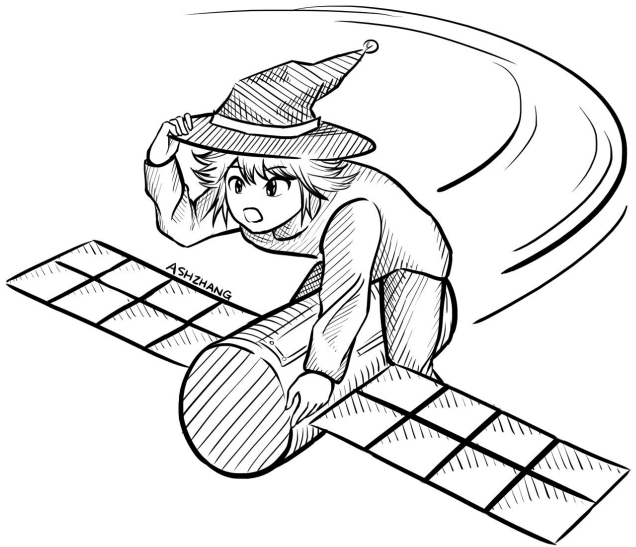
Encryption
Modes of Operation

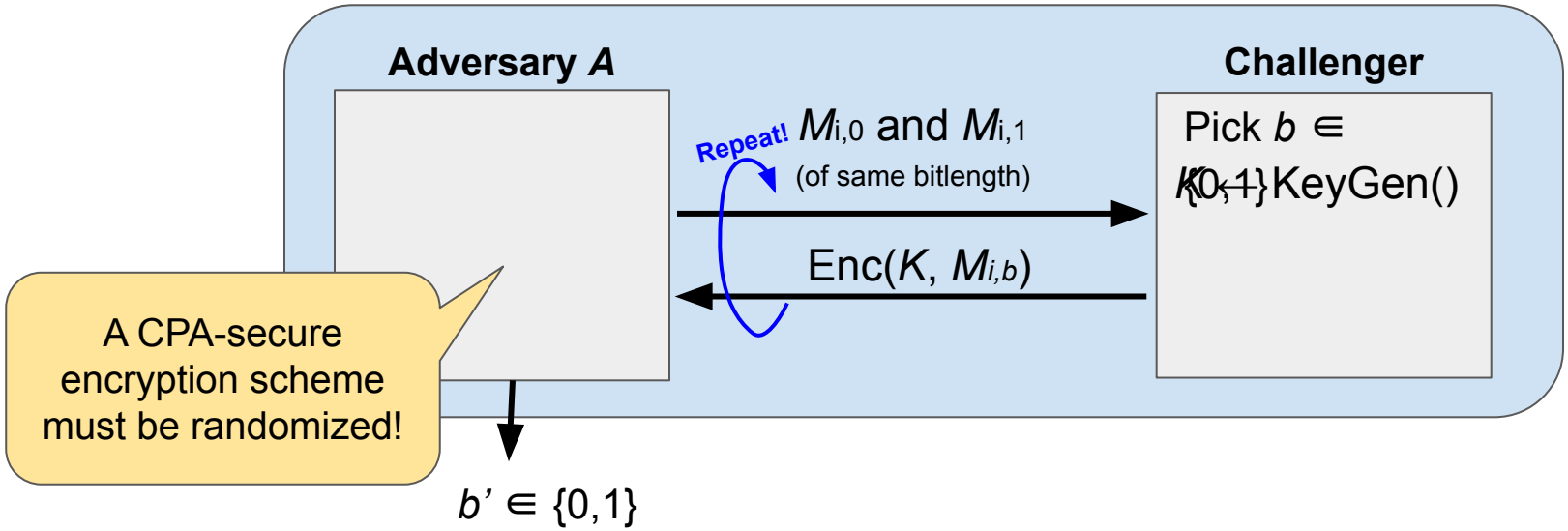
CS 161 Spring 2026 - Lecture 8



Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security – Counter-mode encryption – CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem

Recall: Semantic security for many-time key ("CPA security")



An encryption scheme is *CPA secure* if,
for all "efficient" adversaries A, $|\Pr[A \text{ outputs } b] - ½| \leq$ "negligible."

Recap: Pseudorandom function (PRF)

A pseudorandom function with:

- key space **K**, // e.g., $\mathbf{K} = \{0,1\}^{256}$ 256-bit key
- input space **X**, and // e.g., $\mathbf{X} = \{0,1\}^{128}$ 128-bit input
- output space **Y** // e.g., $\mathbf{Y} = \{0,1\}^{128}$ 128-bit output

is an efficient deterministic algorithm $F : \mathbf{K} \times \mathbf{X} \rightarrow \mathbf{Y}$.

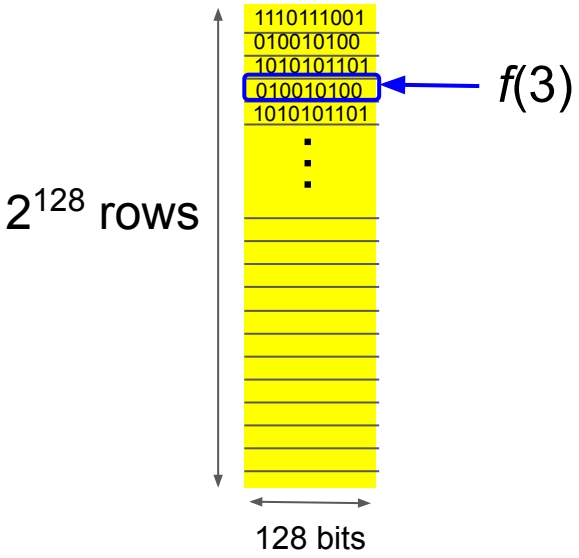
PRF Security:

- Informally: For a random key k, $F(k, \cdot)$ should "look like" a truly random function from **X** → **Y.**

Truly random function vs. Pseudorandom function

Random function *f*

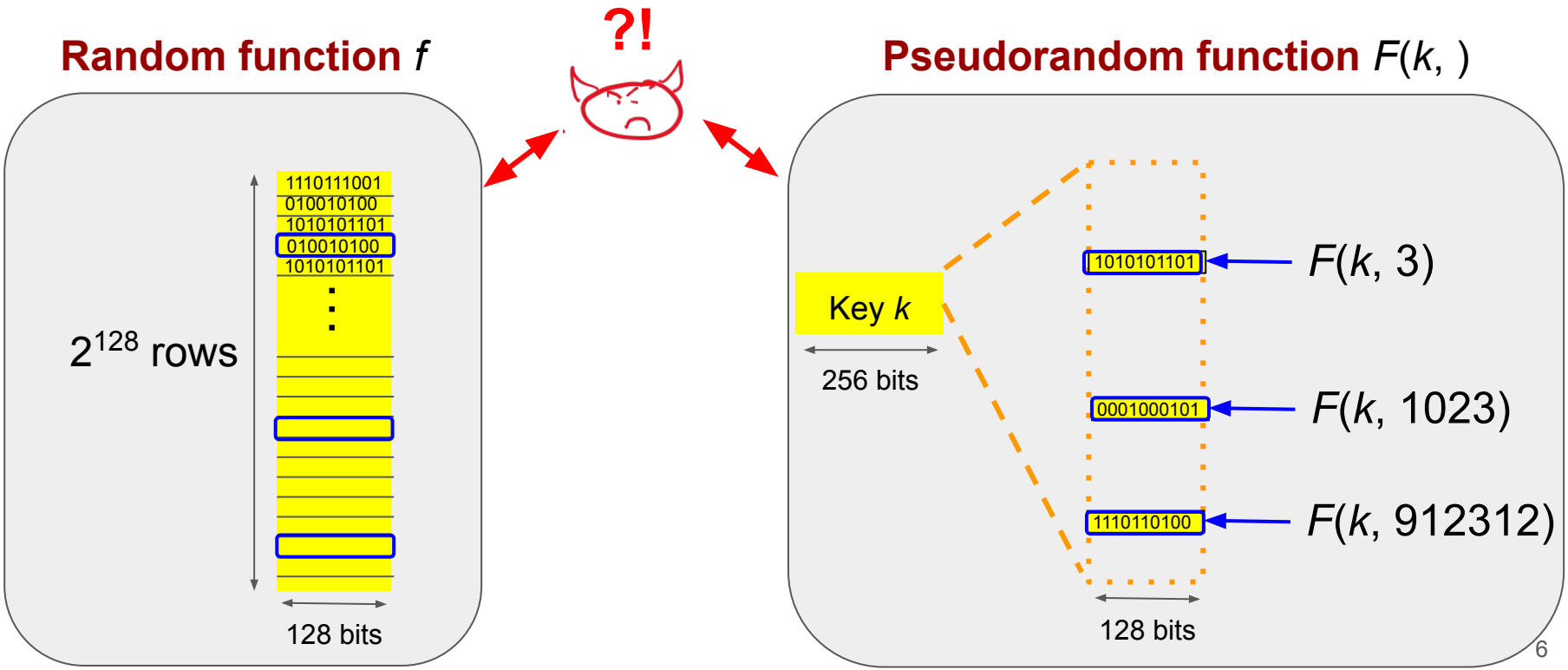
With 128-bit input and 128-bit output



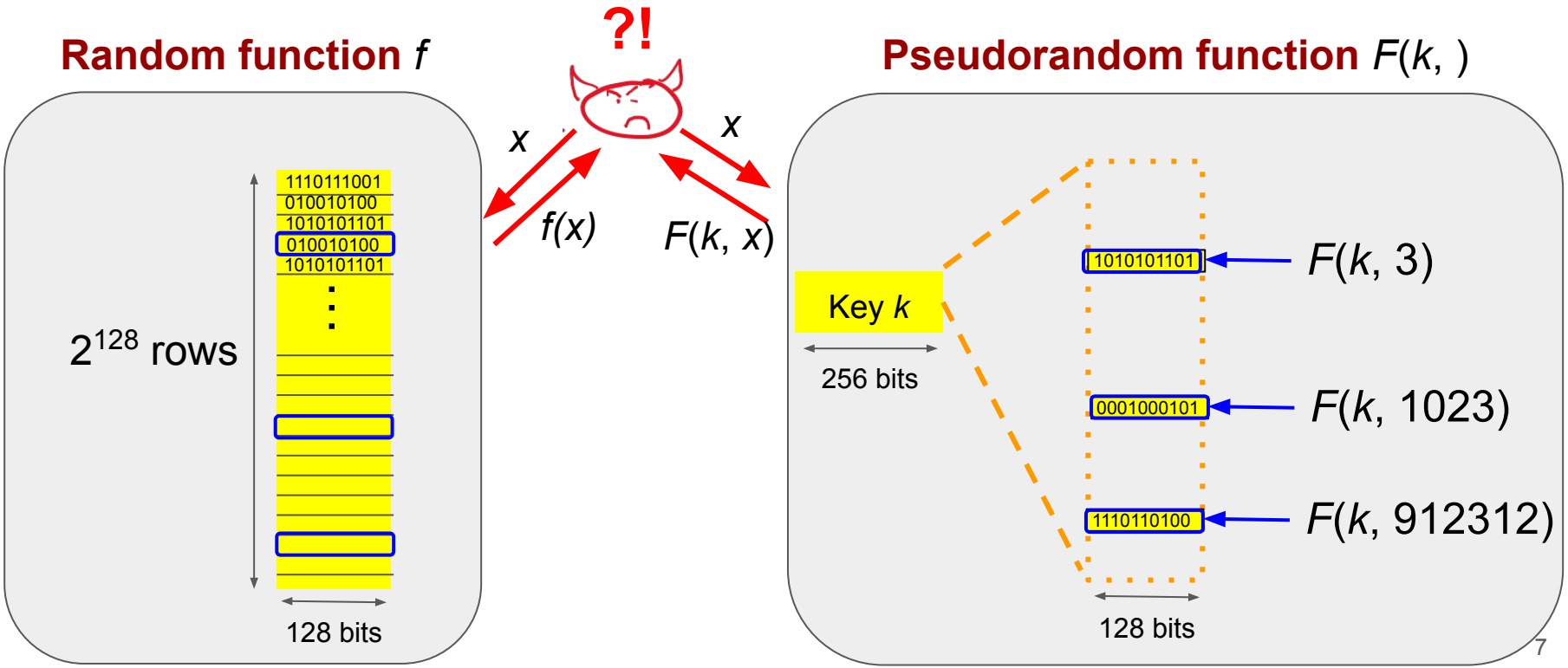
Pseudorandom function *F*

With 256-bit key, 128-bit input, and 128-bit output

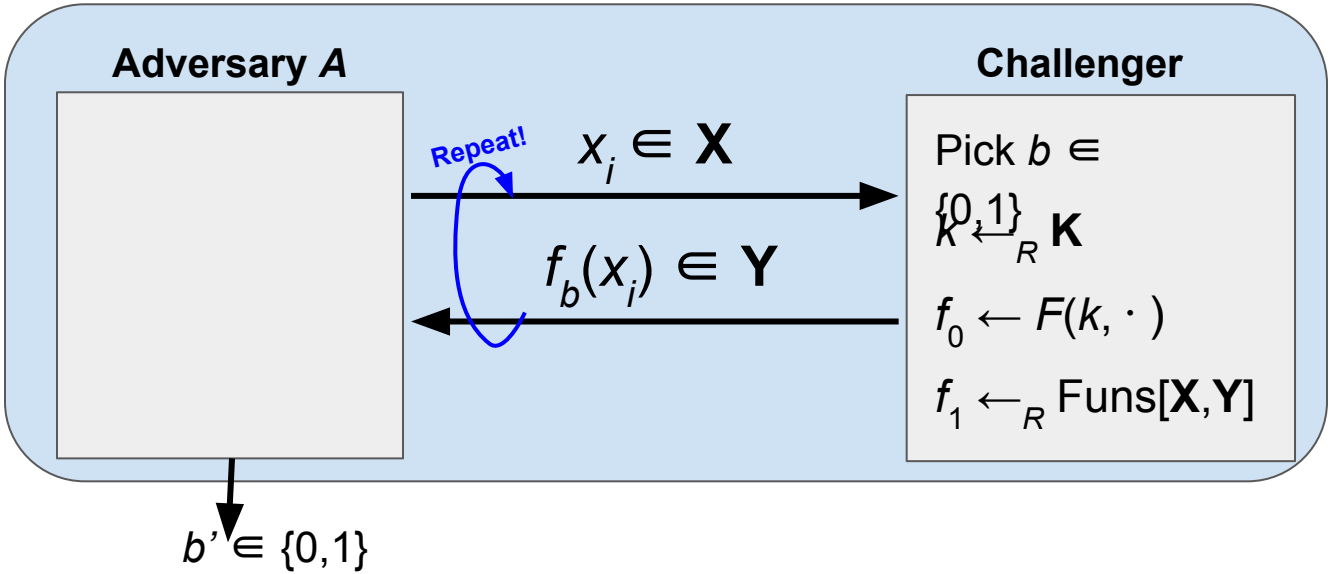
PRF Security: Intuition



PRF Security: Intuition



Recap: PRF Security



A function F : $\mathbf{K} \times \mathbf{X} \rightarrow \mathbf{Y}$ is a *secure pseudorandom function* if,
for all "efficient" adversaries *A*, $|\Pr[A \text{ outputs } b] - ½| \leq$ "negligible."

Recap: Constructing PRFs

- We do not know how to prove that any function is a secure PRF
- Instead, we assume that certain functions behave like PRFs
  - We build confidence in these assumptions through years of cryptanalysis

- AES = U.S. Govt standardized block cipher
  - We assume that it behaves like a PRF
  - **Key size:** 128, 192, or 256 bits
  - **Input/output size:** 128 bits
  - Very fast: 10 GB/sec encryption on modern laptop
- In theory we can build PRFs under “nice” hardness assumptions
  - E.g., that factoring big integers is hard
  - PRFs built this way are 1000x slower than AES → Not used in practice

Counter-mode encryption

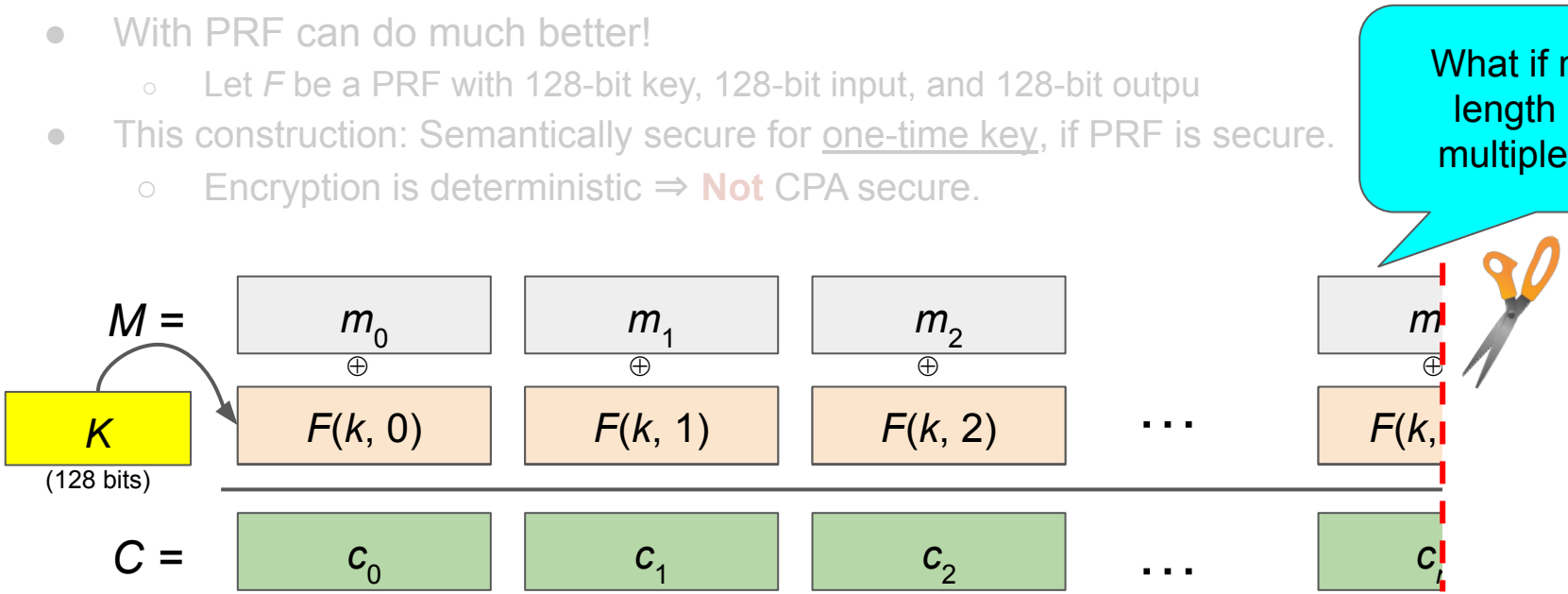
(Using a PRF to encrypt)



PRF application: One-time pad with short key

("Deterministic counter-mode encryption")

- Say your message *M* consists of *n* x 128-bit blocks
- One-time pad needs 128*n* bits of key
- With PRF can do much better!
  - Let *F* be a PRF with 128-bit key, 128-bit input, and 128-bit outpu
- This construction: Semantically secure for one-time key, if PRF is secure.
  - Encryption is deterministic ⇒ **Not** CPA secure.



(Example from Dan Boneh's CS255 slides.)

PRF application: CPA-secure encryption with short key

("Randomized counter-mode encryption", "CTR mode")

- Say you want to send many messages *M* consisting of *n* x 128-bit blocks
- Would like a "fresh" one-time-pad key for each message
- With PRF we can do this!
  - Let *F* be a PRF with 128-bit key, 128-bit input, and 128-bit output

KeyGen() := Output random 128-bit string (i.e., a PRF key)

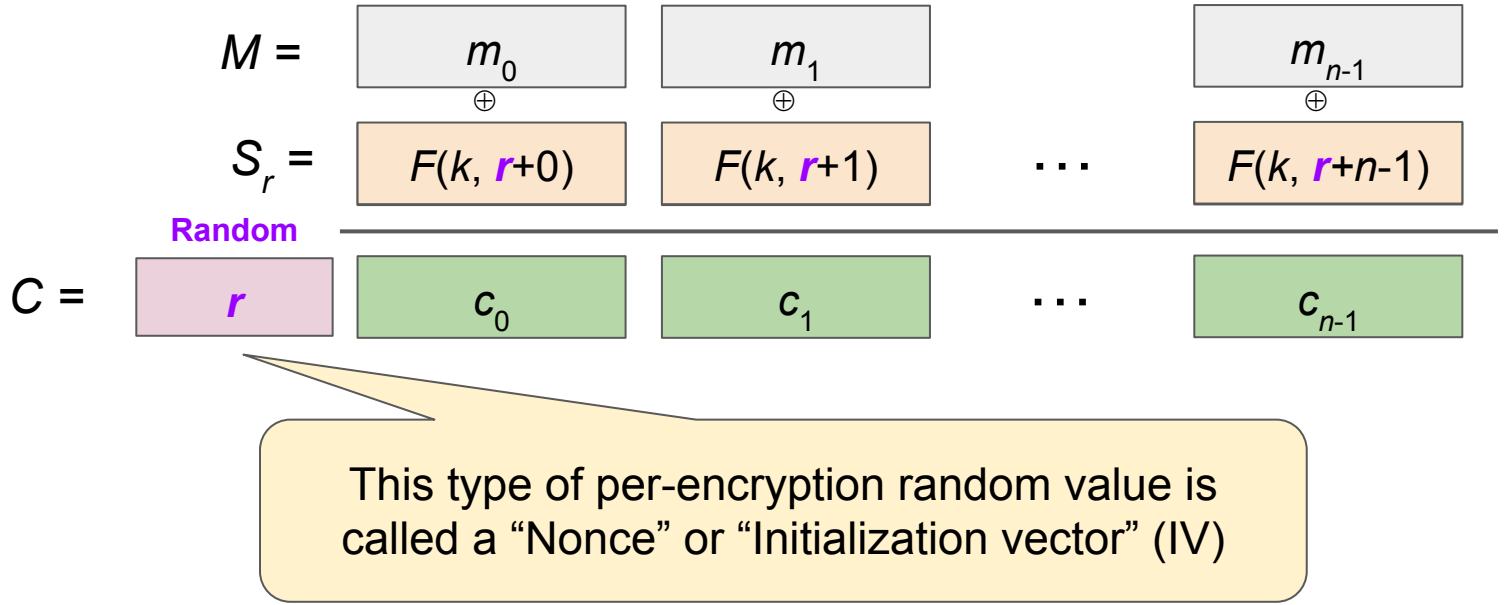
Enc(k, M) := – Choose random 128-bit r (i.e., input to PRF)
– Generate $|M|$-bit string as S_r = [$F(k, r)$ || $F(k, r + 1)$ || $F(k, r + 2)$ || …]
– Output (r, $M \oplus S_r$) as ciphertext

Dec(k, (r, c)) :=
– Generate $|M|$-bit string as S_r = [$F(k, r)$ || $F(k, r + 1)$ || $F(k, r + 2)$ || …]
– Output $c \oplus S_r$ as plaintext

PRF application: CPA-secure encryption with short key

("Randomized counter-mode encryption")

Enc(*k, M)* in picture form…



Why is randomized counter mode CPA secure?

- Security proofs are a way to build confidence in cryptosystems
  - "If *F* is a secure PRF ⇒ randomized CTR mode is CPA secure"
  - We won't cover proofs in this class (take CS276)
- The intuition
  - One-time pad is secure if encryptor uses a fresh random key for each message
  - If for a given key *k*, Enc() never uses the same value $F(k, i)$ twice, each pad S_r used in encryption will be "fresh pseudorandom pad"
  - By PRF security, a pseudorandom pad is as good as random

Question: Is CTR mode CPA secure if we fix the nonce to a constant (e.g., 1234)?

Counter-mode encryption is used everywhere

Why people like counter mode:

- It's relatively simple!
- It's fast: Can encrypt many blocks in parallel.
- Can encrypt messages of any length (don't need to be multiples of 128 bits)

Counter mode is the core of the most common encryption schemes:

- AES-GCM – Used for most HTTPS traffic
- Salsa20/ChaCha20 – Used for SSH, …

Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption – CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem

CBC-Mode Encryption

(The old-school way to encrypt with a block cipher)

Modern way:
– Treat block cipher as a PRF
– Use randomized counter mode

Definition: Block cipher (a.k.a. pseudorandom permutation)

A block cipher with:

- key space **K** and // e.g., $\mathbf{K} = \{0,1\}^{256}$ 256-bit key
- input space **X** // e.g., $\mathbf{X} = \{0,1\}^{128}$ 128-bit "block size"

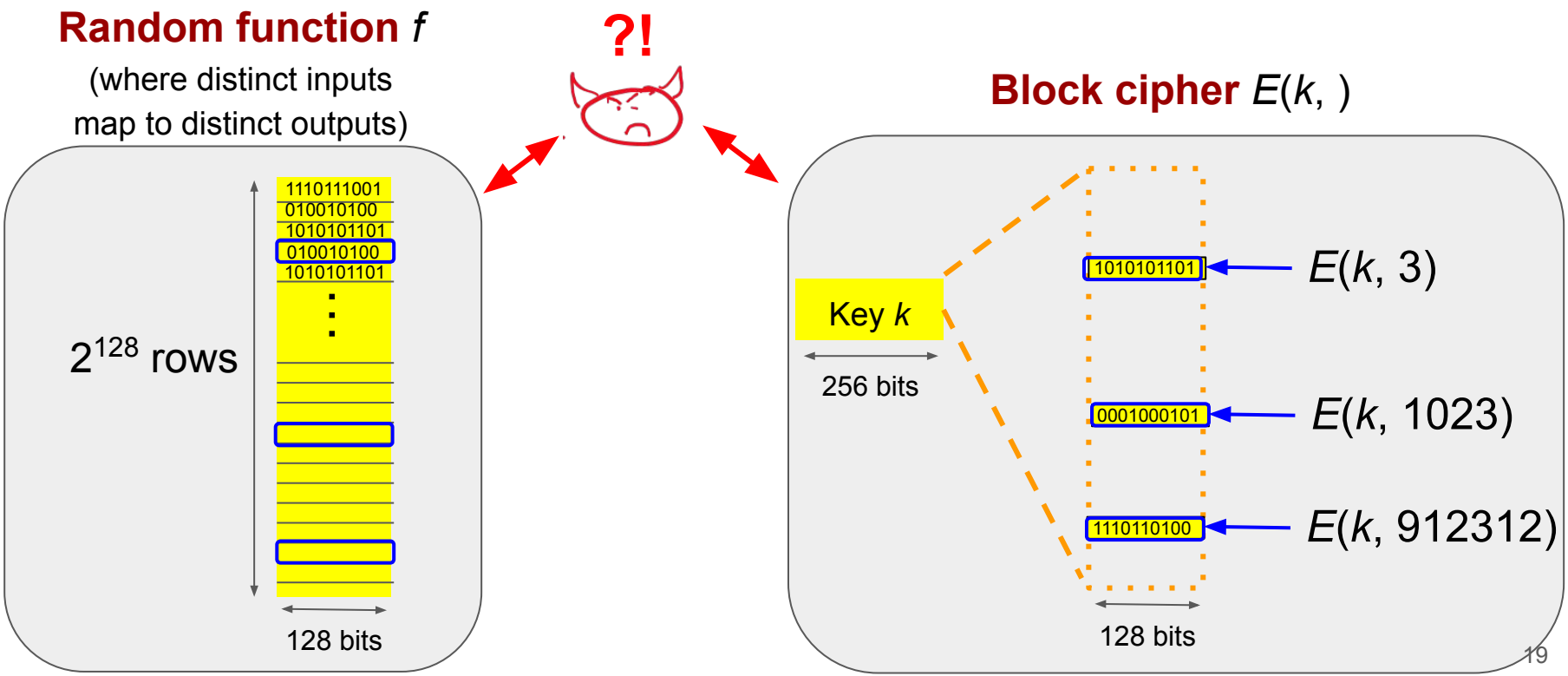
is a pair of efficient deterministic algorithms $E, D : \mathbf{K} \times \mathbf{X} \rightarrow \mathbf{X}$ ("encode," "decode")

Required properties

1. **Distinct inputs map to distinct outputs under *E*:** ($E(k,)$ is "permutation")
   For all keys $k \in \mathbf{K}$, for all inputs $x \neq x' \in \mathbf{X}$: $E(k, x) \neq E(k, x')$

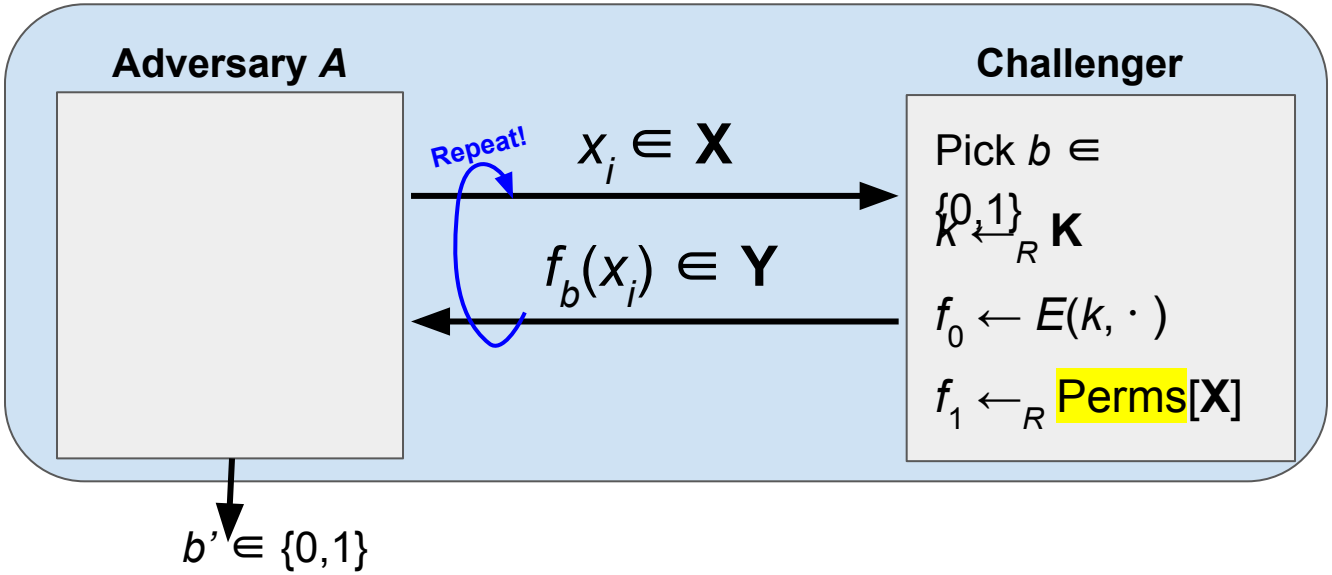
2. ***D*(*k*, ·) computes the "inverse" of *E*(*k*, ·)**
   For all keys $k \in \mathbf{K}$, for all inputs $x \in \mathbf{X}$, $D(k, E(k, x)) = x$.

Block-cipher security: Intuition



Block-cipher security:

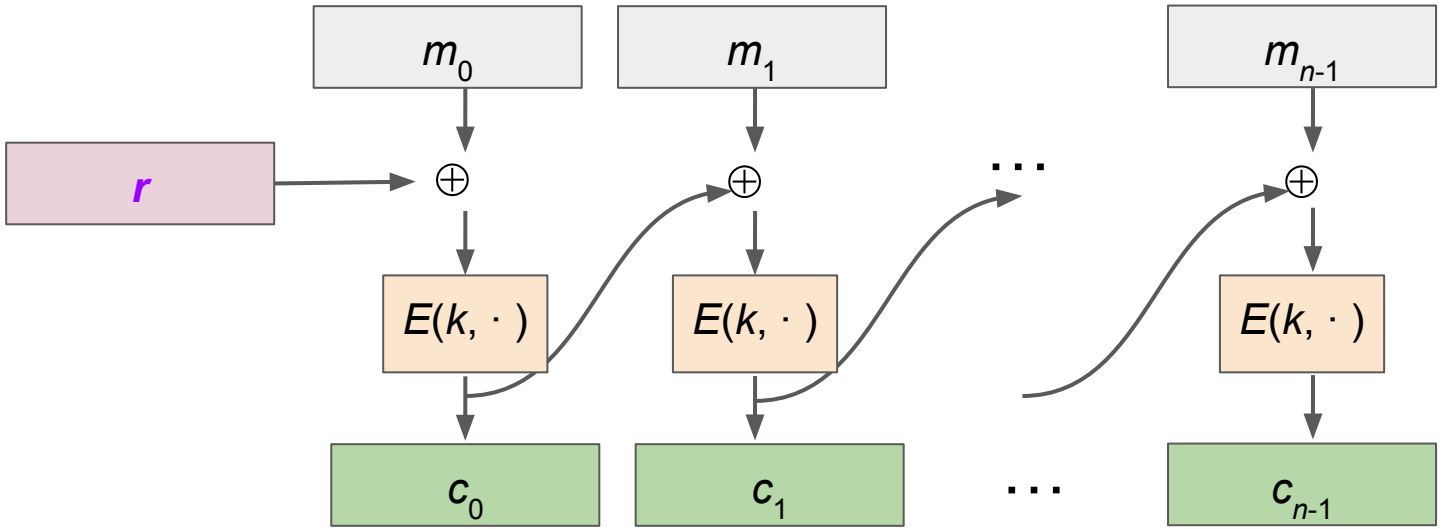
Same as PRF security except $E(k, \cdot)$ should be indist from random permutation



A function F : $\mathbf{K} \times \mathbf{X} \rightarrow \mathbf{Y}$ is a *secure block cipher* (PRP) if,
for all "efficient" adversaries *A*, $|\Pr[A \text{ outputs } b] - ½| \leq$ "negligible."

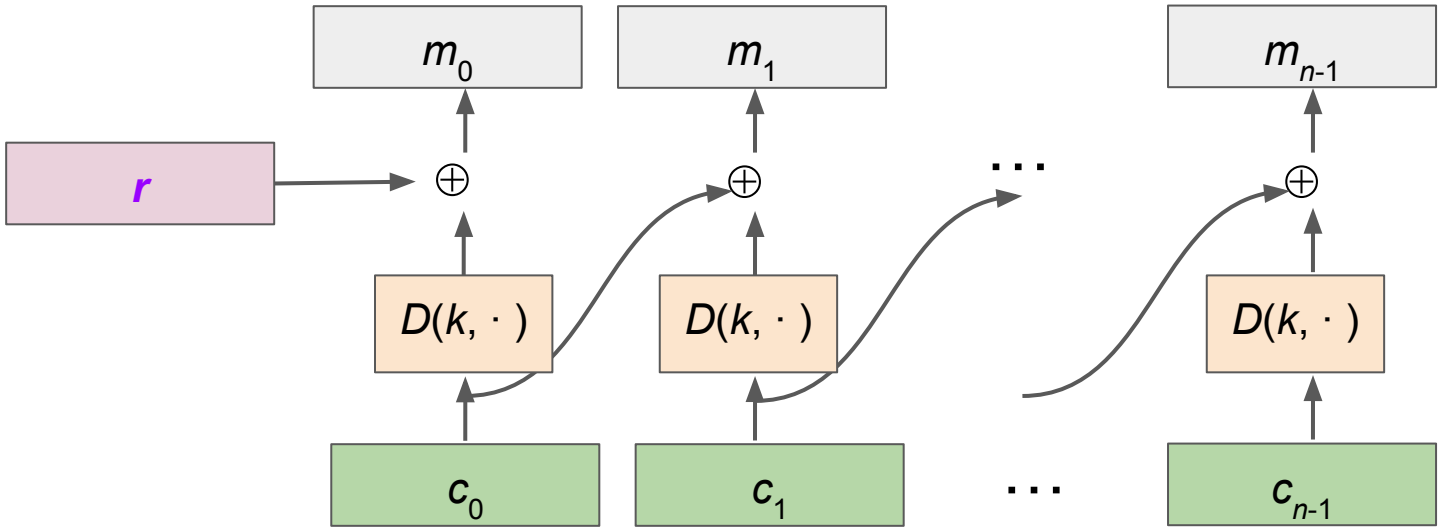
Cipher-Block Chaining Mode (CBC): Encryption

- “Classic” way to encrypt with a block cipher.
- Used in legacy versions of common protocols: TLS, SSH, …



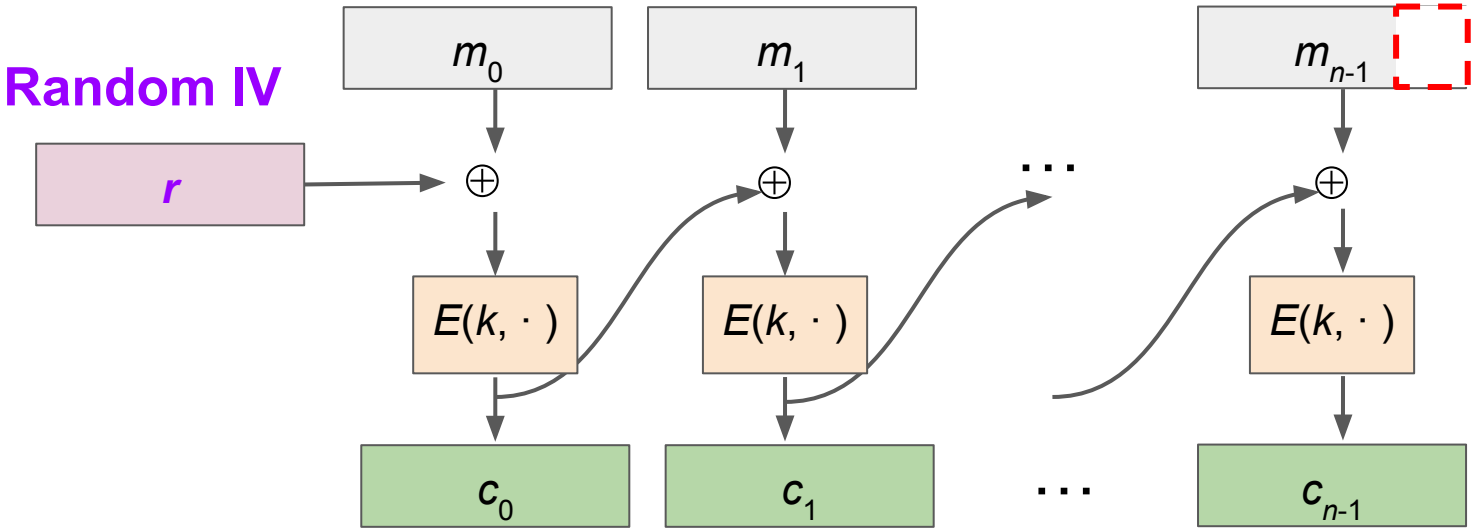
Cipher-Block Chaining Mode (CBC): Decryption

- Decryption just runs the encryption process backwards
- Uses the block cipher decode algorithm $D(k, \cdot)$



Problems with CBC mode

- Encryption is **sequential**: Have 20 CPU cores? Too bad!
- Must **pad message** to multiple of block size—endless fountain of security issues!



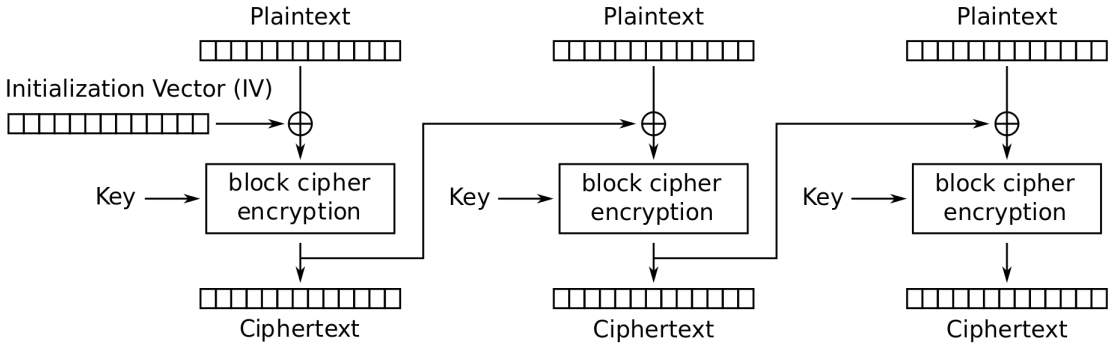
CBC mode with random IV/nonce is CPA secure

- Not obvious at all
- Proof is out of scope
- But you should probably be using counter mode anyways!

Question: What happens if you use the same (key, nonce) pair twice?

CBC Mode: Padding

- What if you want to encrypt a message that isn't a multiple of the block size?
  - AES-CBC is only defined if the plaintext length is a multiple of the block size
- Solution: Pad the message until it's a multiple of the block size
  - **Padding**: Add dummy bytes at the end of the message until it's the proper length



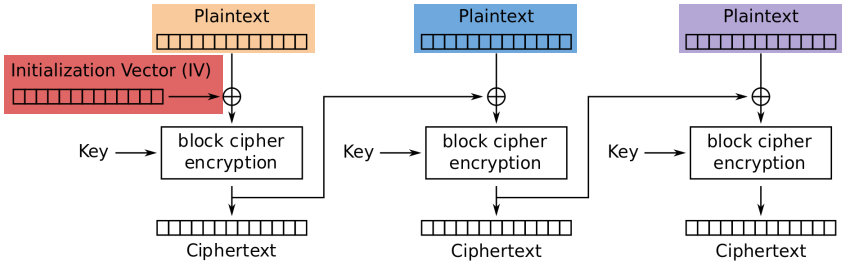
Cipher Block Chaining (CBC) mode encryption

CBC Mode: Padding

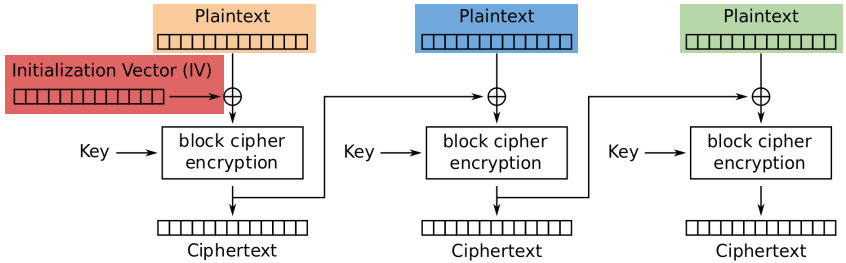
- What padding scheme should we use?
  - Padding with 0 bits?
    - Doesn't work: What if our message already ends with 0's?
  - Padding with 1 bits?
    - Same problem
- We need a scheme that can be unpadded without ambiguity
  - One scheme that works: Append a 1 bit, then pad with 0 bits
    - If plaintext is multiple of n, you still need to pad with an entire block
  - Another scheme: Pad with the number of padding bytes
    - So if you need 1 byte, pad with **01**; if you need 3 bytes, pad with **03 03 03**
    - If you need 0 padding bytes, pad an entire dummy block
    - This is called PKCS #7

CBC Mode: IV Reuse

- Consider two three-block messages: $P_1P_2P_3$ and $P_1P_2P_4$
  - The first two blocks are the same for both messages, but the last block is different
  - What if we encrypt them with the same IV?
- When the IV is reused, CBC mode reveals when two messages start with the same plaintext blocks, up to the first different plaintext block



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode encryption

Reminder: CPA security does not guarantee integrity!

- If you want to protect against tampering, you need more than CPA security
- Next time: How to use PRFs to protect message integrity too



Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem